

정규표현식 (Regular Expression)의 이해와 활용

문자열 패턴 매칭의 강력한 도구

목차

01 정규표현식이란?

- 개념과 특징
- 형식 언어
- 다양한 활용 분야
- 이론적 배경

02 기본 문법 (1)

- 리터럴 문자
- 메타문자
- 점(.) 문자
- 반복 수량자

03 기본 문법 (2)

- 반복 횟수 지정
- 문자 집합/범위
- 부정 문자 집합
- 위치 지정자

04 기본 문법 (3)

- 숫자 관련 문자
- 단어 관련 문자
- 공백 관련 문자
- 그룹화와 캡처

05 고급 기능

- Non-greedy 수량자
- 전방/후방 탐색
- 역참조
- 이름 있는 그룹

06 활용 예제

- 이메일 검증
- 전화번호 검증
- HTML 태그 제거
- 언어별 차이점

정규표현식 이란?

정규표현식의 개념

- 문자열 안에서 특정 패턴을 찾거나 치환하거나 검증할 때 사용하는 도구
- '특정 규칙을 가진 문자열의 집합'을 표현하는 형식 언어

정규표현식의 특징

- 대부분의 프로그래밍 언어와 도구에서 지원
 - 정규 언어 이론에서 유래했으나 현대 구현체는 더 많은 기능 제공
-

기본 문법

리터럴 문자

- 문자 그대로 매치되는 일반 문자
 - 예: 'a'는 문자 'a'와 매치
- 특수문자는 이스케이프(\) 필요
 - 예: \'는 마침표와 매치

메타문자

- 특별한 의미를 가진 문자
- 예: ., *, +, ?, {}, [], ^, \$
- 패턴 매칭의 규칙을 정의
 - 정규식의 핵심 요소

점(.) 문자

- 줄바꿈 문자를 제외한 아무 문자 하나와 매치
 - 예: 'a.b'는 'acb', 'a1b' 등과 매치
 - 's' 플래그 사용 시 줄바꿈도 포함
- 가장 많이 사용되는 메타문자 중 하나

반복 수량자

- * : 0번 이상 반복 ($ab^*c \rightarrow ac, abc, abbc...$)
- + : 1번 이상 반복 ($ab^+c \rightarrow abc, abbc...$)
- ? : 0번 또는 1번 ($colou?r \rightarrow color, colour$)
 - 패턴의 반복 횟수를 지정하는 문자

기본 문법

$\{n\}, \{n,\}, \{n,m\}$

정확한 반복 횟수를 지정하는 수량자

$[]$ (문자 집합)

대괄호 안의 문자 중 하나와 매치

$[^{\dots}]$ (부정 집합)

대괄호 안에 없는 문자와 매치

$^$ (시작 위치)

문자열의 시작 위치를 나타냄

$\$$ (끝 위치)

문자열의 끝 위치를 나타냄

기본 문법

\d, \D

- \d: 숫자 문자와 매치 [0-9]와 동일
- \D: 숫자가 아닌 문자와 매치 [^0-9]와 동일
 - 예: '\d{3}'은 '123'과 매치
 - 전화번호, 우편번호 등에 유용

\w, \W

- \w: 단어 문자와 매치 [A-Za-z0-9_]와 동일
- \W: 단어 문자가 아닌 것과 매치
 - 예: '\w+'는 'abc123'과 매치
 - 변수명, 식별자 검증에 유용

\s, \S

- \s: 공백 문자와 매치 (스페이스, 탭, 줄바꿈)
- \S: 공백이 아닌 문자와 매치
 - 예: '\s+'는 연속된 공백과 매치
 - 텍스트 파싱, 공백 처리에 유용

() (그룹화)

- 패턴을 그룹화하고 캡처하는 기능
- 예: '(ab)+'는 'ab', 'abab'와 매치
- 캡처된 그룹은 나중에 참조 가능
- 복잡한 패턴 구성의 기본 요소

정규표현식의 고급 기능

정규표현식의 기본 문법을 넘어서는 고급 기능들은 더 복잡한 패턴 매칭을 가능하게 합니다. 이러한 고급 기능들은 언어나 구현체에 따라 지원 여부가 다를 수 있으니 사용 전 확인이 필요합니다.

Non-greedy/lazy 수량자

- `*?`, `+?`, `??` 등의 형태로 사용
- 기본 수량자(`*`, `+`, `?`)는 최대한 많이 매치(greedy)
- lazy 수량자는 가능한 최소한으로 매치

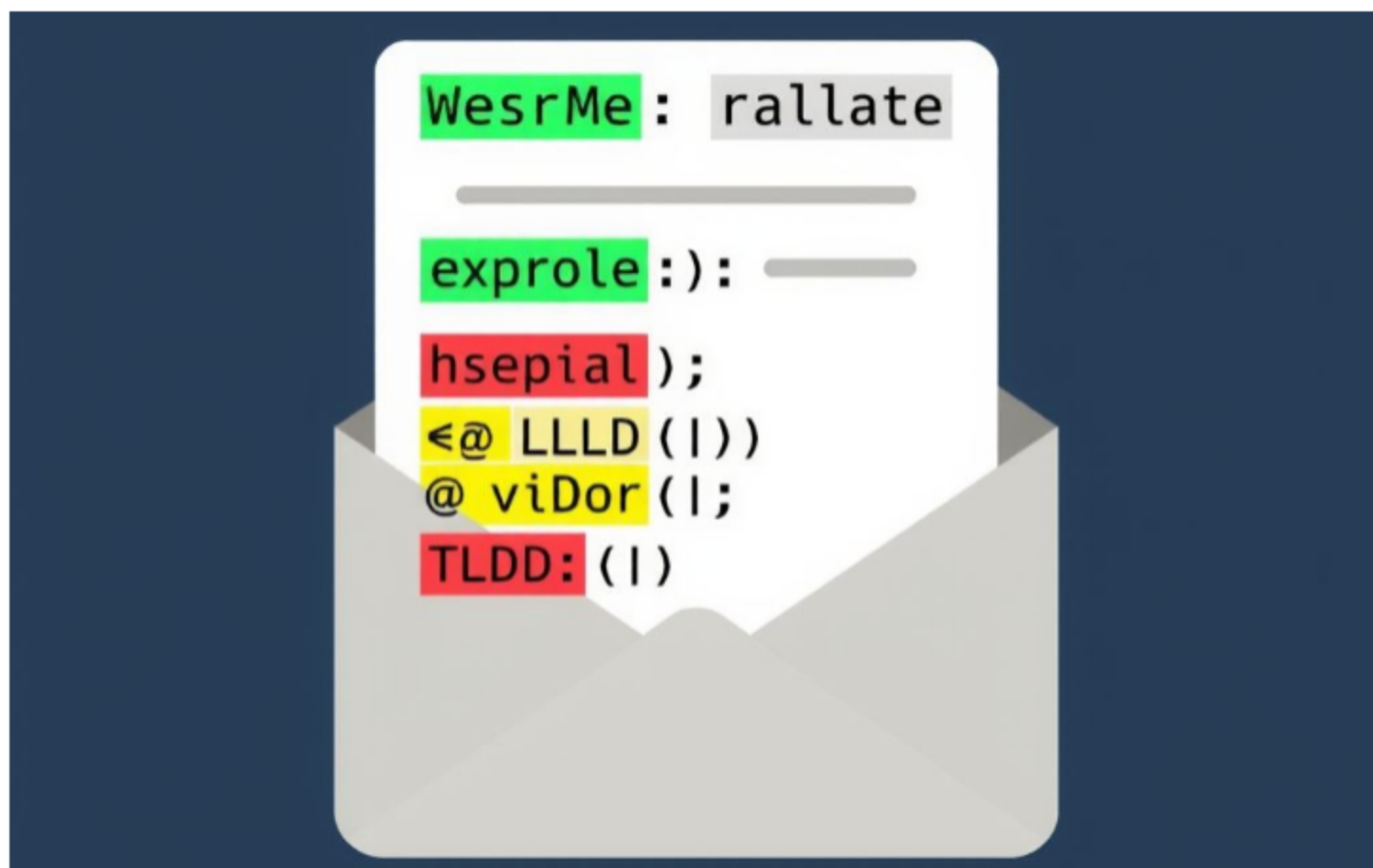
Lookahead/Lookbehind

- 패턴 주변 조건 확인 (매치에는 포함 안 됨)
- 전방탐색: `(?=긍정)`, `(?!부정)`
- 후방탐색: `(?<=긍정)`, `(?<!부정)`

Backreference와 Named Groups

- 역참조: 이미 매치된 그룹을 재사용 (`\1`, `\2`)
- Named Groups: 그룹에 이름 부여
- 예: `(?<name>패턴)` 또는 `(?P<name>패턴)`

정규표현식 활용 예제



이메일 & 전화번호 검증

- 이메일 검증 패턴:

```
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

- 전화번호 검증 패턴(한국식):

```
^\d{2,3}-\d{3,4}-\d{4}$
```

HTML;

Al celsises forfhclwase for mplewe, tald we tag ft's:
sispens.ent I cahlltweef for rellewt, anid well'ls fthe
code: or regex mbital content, at HTML.tags)
otltens.ent I cahlltweef after an recites. >>

HTML;

Al celsises forfhclwase for mplewe, tald weltas fthe
strix; for regex >>

sispens.ent I cahllitwase for (ittl, retal) to tags tagis I tase
recrex: HTML.. Al Ath un fochtag:
sispens.ent I cahlltweef for reblewe, anid well'ls fthe
revalion.ay>

HTML 태그 제거

- HTML 태그 제거 패턴:

```
<[>]+>
```

- 이 패턴은 '<'로 시작해서 '>'로 끝나는
모든 문자열을 찾아 제거합니다.

정규표현식 사용 시 주의사항

탐욕성(Greedy) vs 최소 매치(Lazy)

문법 차이 (flavor)

- 언어/라이브러리마다 정규표현식 기능이 조금씩 다름
- JavaScript, Python, Java, .NET 등에서 지원하는 기능이 다름
- 특히 lookbehind, 역참조 기능이 제한적인 구현체도 있음

가독성과 유지보수

과도한 복잡성 & 성능 문제

- 잘못 작성된 정규표현식은 입력 길이가 커질수록 처리 비용이 폭발적으로 증가
- 특히 backtracking 기반 구현에서는 ReDoS(정규표현식 서비스 거부 공격) 위험
- 복잡한 중첩 패턴은 성능 저하의 원인

테스트 필수

Unicode & 문자 인코딩

- 한글, 특수문자 등 유니코드 범위를 다룰 때는 정규표현식 엔진의 유니코드 지원 확인 필요
- 문자 클래스가 어떻게 해석되는지 확인
- 인코딩에 따라 동작이 달라질 수 있음

JavaScript의 정규표현식

```
6      eyernewaif:: fitlte; (FLLIZ7LEARNA(FS_polt)):
10
17      Phone putcrc (eclank; (irfalcS60Bity-Snatif2_omam)):
18      nejich one:
17          nore fatee: (fone plass.cul))
18 :      nore matching: (intlearry-adSBAllfe.ripsa)25_prod)):
19      }
10      tate: // patttern Matlcas:
14      noge: numbers: tast:mnonal_camay_proa)):
19      paterNative cortacticmals.up//7.coct samallep, #7.nes/antioders
17      isten paltiaft: fections.cly: {
```

기본 문법과 특징

- /패턴/플래그 형태로 사용 (예: /abc/i, /\d+/g)
- 주요 플래그: g(global), i(ignore case), m(multiline), s(dotAll), u(Unicode), y(sticky)

주요 기능과 예제

- ES2018 이후 Lookbehind, Named groups 지원
- 예제: const regex = /(?!<area>\d{2,3})-(?!<num>\d{3,4})-(?!<last>\d{4})/;

Python과 Java의 정 규표현식

Python

- re 모듈 사용
- 형식: `re.match(pattern, string)`
- Named groups: `(?P<name>...)`
- 플래그: `re.IGNORECASE`, `re.MULTILINE`
- Verbose mode(`re.VERBOSE`): 공백/주석 허용
- Unicode 지원 강력 (`\w`, `\d` 등도 유니코드 대응)
- 예: `import re`
`pattern = r'(?P<area>\d{2,3})-(?P<num>\d{3,4})-(?P<last>\d{4})'`



Java

- `java.util.regex` 패키지 사용
- `Pattern` + `Matcher` 객체 조합
- 플래그: `Pattern.CASE_INSENSITIVE` 등
- Named groups: `(?<name>...)`
- Unicode 클래스 지원 (`\p{L}`, `\p{Digit}`)
- Atomic grouping `(?>..)` 지원
- 예: `Pattern p = Pattern.compile(regex);`
`Matcher m = p.matcher(input);`

SQL과 다른 언어의 정규표현식 차이

MySQL	PostgreSQL	Oracle	C# (.NET)
- REGEXP 또는 REGEXP_LIKE() - POSIX 정규식 기반 - Lookbehind - Named groups	- ~, ~* (대소문자 무시) 연산자 - POSIX Extended Regex + 일부 확장 - Lookahead 부분 지원	- REGEXP_LIKE, REGEXP_REPLACE, REGEXP_SUBSTR - Perl 스타일에 가까움 - Lookbehind 제한적	- System.Text.RegularExpressions - Perl 호환 + 확장 - Balancing group, Atomic group 지원

정규표현식은 언어/환경마다 구현 방식과 지원하는 기능이 다릅니다. JS, Python, Java, C#은 PCRE 기반으로 고급 기능이 많고, SQL(특히 MySQL, PostgreSQL)은 POSIX 기반으로 상대적으로 단순합니다. Oracle은 SQL 중에서는 가장 강력하게 Perl 스타일을 지원합니다.